

CESAR School — Análise e Desenvolvimento de Sistemas (2026.2)

Infraestrutura de Software · Projeto Integrador

Rota Vital

Processamento paralelo na camada de aplicação

Relatório de cobertura nacional: cruzamento solicitações × estoque com threads

Equipe

Allan Max de Jesus Rodrigues de Lima
Boniek Araujo dos Santos Junior
Caio Cesar Leandro Amorim
Luan Ventura Ferreira de Moura
Miguel Victor Lussac Barboza
Pedro Augusto Carvalho Araujo
Vinicius Pessoa de Albuquerque
Wesley Yuri da Silva

Código: <https://github.com/PCarvalho04/Vital/tree/feature/processamento-paralelo>

Pull request: <https://github.com/PCarvalho04/Vital/pull/1>

Recife, setembro de 2026

1. Justificativa

O Rota Vital nasce como um sistema web em três camadas (apresentação, API e banco de dados). Assumindo a escala nacional — milhares de hospitais, estoque da ordem de milhões de bolsas e painéis consultados o dia inteiro —, buscamos uma operação da camada de aplicação cujo tempo seja dominado por **processamento**, e não por espera de banco ou rede, e que possa ser dividida em partes independentes.

1.1 Como chegamos à operação

Seguimos o roteiro proposto, avaliando as operações que processam volume de dados:

Candidata	Big-O	Gargalo na escala nacional	Particionável?	Decisão
Estatísticas do estoque (contagem por tipo, status, validade)	$O(n)$	Banco — resolve-se com GROUP BY e índices	Sim	Descartada: a resposta certa é SQL
Alocação de uma bolsa para uma solicitação	$O(n \log n)$	Banco (leitura do estoque) e transação	Pouco — operação unitária	Descartada
Deteção de solicitações duplicadas	$O(n)$ com hash	Banco — restrição de unicidade	Sim	Descartada: resolve-se no banco
Cruzamento solicitações × estoque (relatório de cobertura)	$O(S \times B)$	CPU — regras avaliadas bolsa a bolsa	Sim, por solicitação	Escolhida

1.2 A operação escolhida

Relatório de cobertura nacional. Para cada solicitação de sangue registrada, o serviço percorre o estoque e conta quantas bolsas poderiam atendê-la, aplicando as regras do Rota Vital:

- a bolsa está com status DISPONIVEL;
- é do mesmo componente solicitado (hemácias, plasma ou plaquetas);
- está dentro da validade;
- é compatível pelo sistema ABO/Rh com o tipo do receptor — a mesma regra usada na alocação;
- está armazenada a até **150 km** do hospital solicitante (abastecimento regional).

O resultado é agregado por tipo sanguíneo: solicitações, solicitações atendíveis pelo estoque próximo, total de bolsas compatíveis e quantas delas vencem em até 7 dias (prioridade FEFO). É o tipo de painel que a rede consultaria continuamente para decidir remanejamentos entre unidades — com 100 mil bolsas, por exemplo, apenas 87 de 137 solicitações O- têm cobertura regional.

1.3 Complexidade da solução sequencial

Sendo **S** o número de solicitações e **B** o número de bolsas, o algoritmo executa, para cada solicitação, uma varredura completa do estoque: S iterações externas × B iterações internas de custo constante (até cinco verificações), mais a agregação final em $O(8)$. **Complexidade: $O(S \times B)$.** Com $S = 1.000$ e $B = 1.000.000$ são 10^9 verificações por requisição — cerca de 24 segundos em uma única thread.

1.4 Onde está o gargalo

Na escala nacional o custo dominante é o **cálculo**. A leitura do estoque custa $O(B)$ e pode ser mantida em cache, enquanto o cruzamento executa $S \times B$ verificações sobre os mesmos dados — ordens de grandeza a mais. Durante a execução a CPU fica 100% ocupada, sem espera de I/O.

Poderíamos resolver no SQL? Um *join* com as mesmas regras continuaria sendo $O(S \times B)$, só que executado no banco compartilhado por todos os hospitais. E pré-agregar o estoque não ajuda: como a distância depende da unidade que armazena a bolsa e a validade importa dia a dia, a chave de agrupamento seria unidade $\times$ tipo $\times$ componente $\times$ validade — $3.000 \times 8 \times 3 \times \sim 65 \approx 4,7$ milhões de grupos, mais do que o próprio número de bolsas. O trabalho é inerentemente par a par.

Para isolar esse custo nas medições, os dados são gerados em memória (100 mil e 1 milhão de bolsas, 1.000 solicitações, 3.000 unidades de armazenamento e 5.000 hospitais distribuídos pelo território), com semente fixa — toda execução processa exatamente os mesmos dados — e mantidos em cache. O tempo medido no endpoint corresponde, portanto, ao processamento.

1.5 Por que os dados são particionáveis

A contagem de cada solicitação é **independente** das demais: depende apenas da própria solicitação e do estoque, que é somente lido. Assim, a lista de solicitações é dividida em p fatias contíguas, cada thread acumula o resultado da sua fatia em contadores **próprios**, e ao final a thread principal soma as parciais. Como a soma é associativa e comutativa, a ordem de término das threads não altera o resultado; como nenhuma thread escreve em dados compartilhados, não há *locks* nem risco de *race condition*.

2. O serviço

A operação foi implementada como endpoint real na API Spring Boot (Java 21) do Rota Vital: a requisição chega, o servidor processa e a resposta volta em JSON.

GET /api/relatorios/cobertura?bolsas=1000000&threads=8

- threads=1 executa a versão **sequencial**, na própria thread da requisição;
- threads=2, 4, 8 usam um **ExecutorService** com pool fixo de threads de plataforma, uma fatia por thread;
- virtual=true usa **virtual threads** do Java 21 (opcional); solicitacoes define S (padrão 1.000).
- A resposta traz o relatório por tipo sanguíneo, o modo, o número de threads e o tempo de processamento medido no servidor.

2.1 Paralelização

Cada tarefa recebe um intervalo contíguo de solicitações e devolve um objeto `Parcial` próprio (contadores por tipo sanguíneo). A agregação acontece depois que todas as tarefas terminam:

```
try (ExecutorService executor = modo == Modo.VIRTUAL
    ? Executors.newVirtualThreadPerTaskExecutor()
    : Executors.newFixedThreadPool(threads)) {
    List<Future<Parcial>> futuros = new ArrayList<>(threads);
    int total = solicitacoes.size();
    for (int t = 0; t < threads; t++) {
        int inicio = (int) ((long) total * t / threads); // fatia [inicio, fim)
        int fim = (int) ((long) total * (t + 1) / threads);
        futuros.add(executor.submit(() -> processarFatia(bolsas, solicitacoes, inicio, fim)));
    }
    Parcial resultado = new Parcial();
    for (Future<Parcial> futuro : futuros) {
        resultado.somar(futuro.get()); // agregação ao final
    }
    return resultado.paralista();
}
```

2.2 Mesma resposta nas duas versões

O teste `CoberturaEstoqueServiceTest` executa o cálculo sequencial e as versões com 2, 3, 4 e 8 threads (de plataforma e virtuais) sobre a mesma massa e exige resultados **idênticos**. O script de medição repete essa verificação em todas as chamadas ao endpoint: nenhuma divergência foi encontrada, o que confirma a ausência de *race condition*. Os 15 testes do projeto passam.

2.3 Organização do código

Arquivo (backend/rotavital/...)	Papel
controller/RelatorioCoberturaController	Endpoint GET /api/relatorios/cobertura e validação dos parâmetros
service/CoberturaEstoqueService	Versão sequencial, ExecutorService (plataforma) e virtual threads
service/GeradorDadosService	Massa de dados sintética em memória, semente fixa, em cache
model/relatorio/BolsaEmEstoque, DemandaHospitalar	Projeções de leitura com a localização
test/.../CoberturaEstoqueServiceTest	Igualdade sequencial × paralelo e regras de negócio
docs/atividade-threads/medir.py	Script de medição: CSVs e gráfico

3. Medições

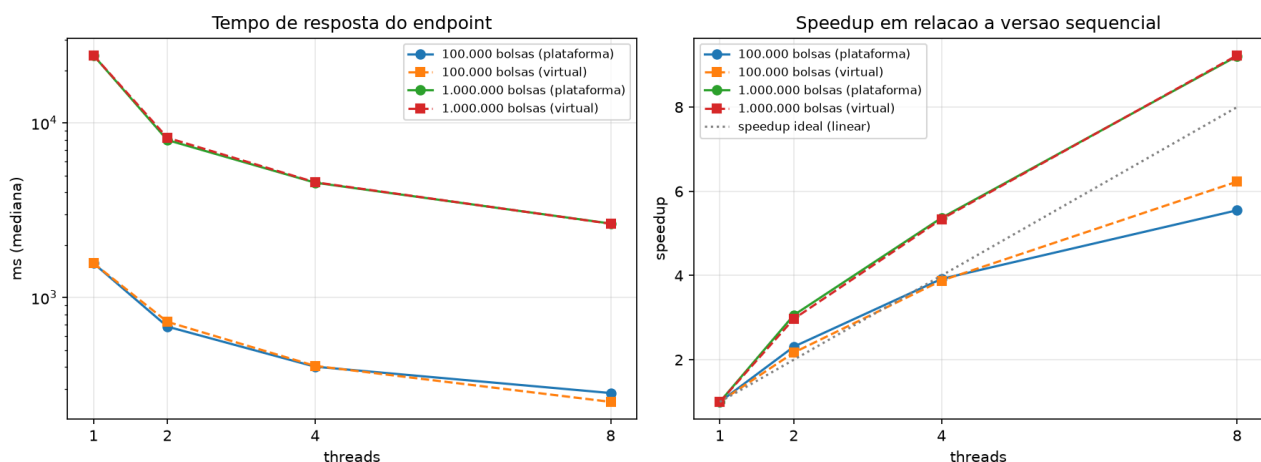
Ambiente: notebook com Intel Core i5-12450HX — processador híbrido com 4 núcleos de desempenho (P, com hyper-threading) e 4 de eficiência (E), 12 threads lógicas, cache L3 de 12 MB — 16 GB de RAM, Windows 11, JDK 21 (Temurin), heap de 4 GB.

Método: para cada tamanho, uma rodada de aquecimento (gera a massa de dados e deixa o JIT compilar todos os caminhos) seguida de 5 rodadas **intercaladas** — cada rodada passa por todas as configurações, para que variações da máquina afetem todas igualmente. Registramos a mediana do tempo de resposta HTTP medido pelo cliente. S = 1.000 solicitações em todos os casos.

Afinidade de CPU: na primeira bateria, sem controle, o Windows colocava a thread sequencial sozinha nos núcleos E: 3.542 ms, contra 1.572 ms num núcleo P — o que inflava artificialmente o speedup. Por isso a medição final restringe o processo Java aos 4 núcleos P (8 threads lógicas), garantindo uma base de comparação justa.

Bolsas	Versão	Threads	Tempo de resposta (ms)	Speedup	Eficiência
100.000	Sequencial	1	1.572	1,00x	100%
100.000	Plataforma	2	681	2,31x	116%
100.000	Plataforma	4	401	3,92x	98%
100.000	Plataforma	8	283	5,55x	69%
100.000	Virtual	2	725	2,17x	108%
100.000	Virtual	4	405	3,88x	97%
100.000	Virtual	8	252	6,23x	78%
1.000.000	Sequencial	1	24.421	1,00x	100%
1.000.000	Plataforma	2	7.985	3,06x	153%
1.000.000	Plataforma	4	4.550	5,37x	134%
1.000.000	Plataforma	8	2.652	9,21x	115%
1.000.000	Virtual	2	8.233	2,97x	148%
1.000.000	Virtual	4	4.574	5,34x	134%
1.000.000	Virtual	8	2.646	9,23x	115%

Speedup = tempo sequencial ÷ tempo paralelo. Eficiência = speedup ÷ número de threads.



Tempo de resposta (escala logarítmica) e speedup por número de threads. A linha pontilhada é o speedup ideal (linear).

4. Análise

O ganho foi linear? Não, e o motivo muda com o tamanho. Com 100 mil bolsas (~7 MB, cabem no cache L3) o speedup seguiu o ideal até 4 threads (3,92x), mas ficou em 5,55x com 8: são só 4 núcleos físicos, o hyper-threading divide as mesmas unidades de execução, e há custos fixos (criar threads, agregar, HTTP). Com 1 milhão (~70 MB) foi *superlinear* (9,2x): o sequencial fica esperando a RAM (10× mais dados custaram 15,5× mais tempo), e as threads, varrendo o mesmo estoque juntas, reaproveitam no L3 compartilhado o que as outras trouxeram.

A Big-O mudou? Não: o trabalho segue $O(S \times B)$; só o tempo cai para $\sim O(S \times B / p)$.

Concorrência × paralelismo. Na Mesa DJ havia **concorrência**: vários eventos independentes intercalados, o que funciona até num só núcleo. Aqui é **paralelismo**: uma tarefa dividida em fatias que rodam ao mesmo tempo em núcleos diferentes. Por isso as virtual threads empataram com as de plataforma (9,23x × 9,21x): elas ajudam quem espera I/O, não quem calcula.

E quando nem 8 threads bastarem? (1) Gerar o relatório de forma assíncrona quando o estoque muda e servi-lo do cache; (2) escalar horizontalmente com várias instâncias da API atrás de um balanceador; (3) distribuir o cálculo: as fatias viram mensagens numa fila (Kafka/RabbitMQ), processadas por *workers* em várias máquinas, com agregação no final. É a mesma ideia desta atividade, particionar e agregar, levada de threads num processo para muitas máquinas.